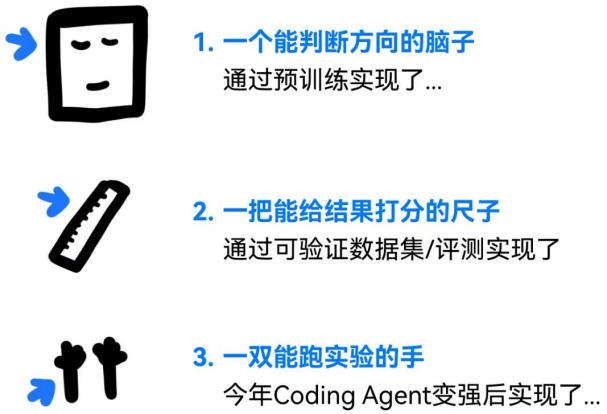


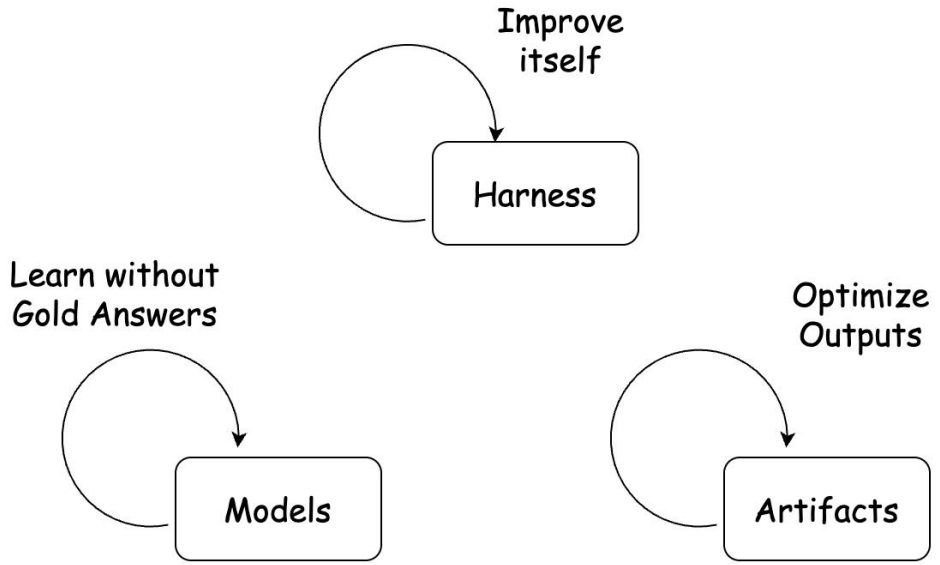
Recursive Self Improving Agent for AI infrastructure

肖翊成 2026.10.8

Self-Evolving / RSI

- 本质就是指 AI 的 自进化，即用AI来去迭代AI的下一个版本
- 这个概念听起来其实非常老旧，并不是一个很新的概念，毕竟我们很早已经在使用 AI 在写代码、跑实验然后来优化我们的 AI 模型、写我们的 AI应用了
- 利用 Shilong Liu的分类标准，RSI可以被分为以下三个门类
  - Artifacts 的 RSI
  - Harness 的 RSI
  - Models 的 RSI

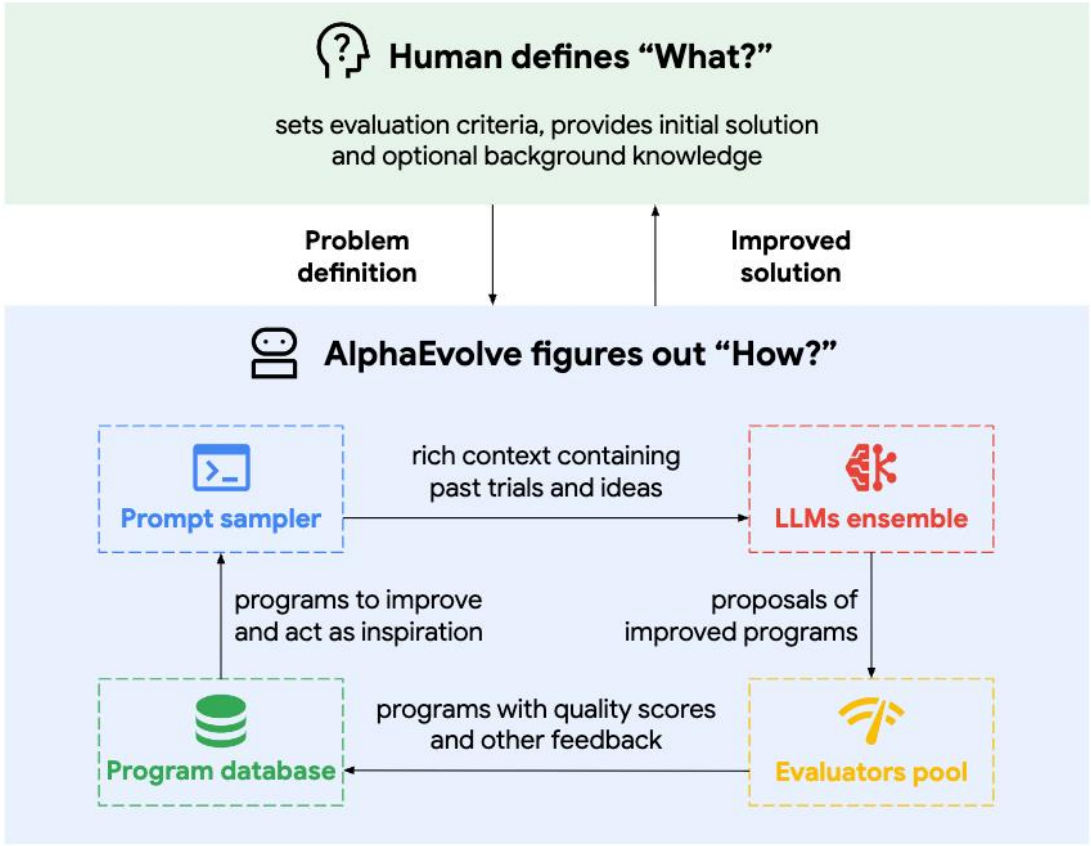




https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/

Artifacts Iterative Optimization

- Artifacts 在这里指代码/论文/算法
- 不涉及修改模型权重，不涉及修改Agent自身的脚手架
- 这种RSI的例子：
  - Auto-Research
  - Kernel Optimization
  - Algorithm Discovery
- 需要人来设定「目标」和「评估方式」
- 「The world is the agents'oyster.」交由模型来探索所有的可能



https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/

Agent Harness Self-improvement

- Harness 本身的定义也非常的广泛
- 不修改模型权重，仅对Agent的**执行框架（脚手架）**进行更新
- **Prompt Learning and Memory：**存储下来有用的Rules在Prompt（GEPA）、在Context Playbook（ACE）和Memory（Mem0）里面
- **Tool and Skill Creation：**把固定的操作用Tool/Skill的方式固定下来
- **Towards Multi-Agent Self-Evolving：**用特定的Expert来管理 Context，如何Routing

https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/

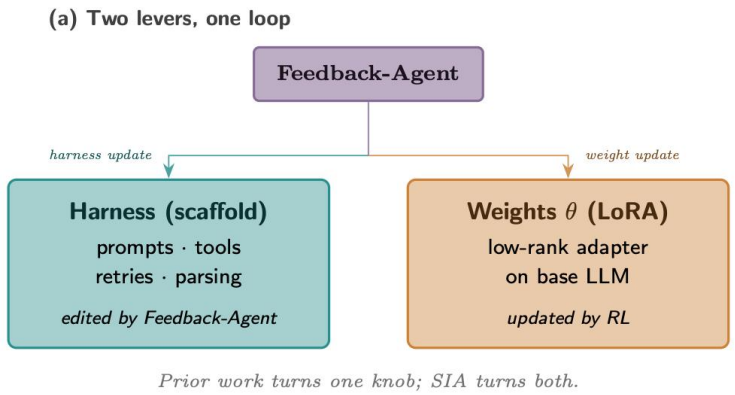
Model Learning without Gold Answers

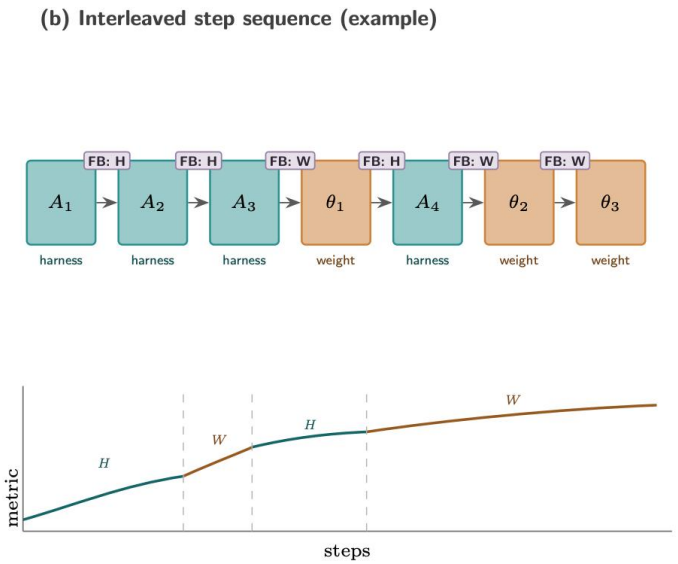
- 这个范式下其实包括很多概念
  - Self-training, Weak Supervision, Self-play, Reinforcement Learning, Test-time Training, Online Learning, Continual Learning
- 直接修改模型权重
- 很多时候我们是没有Ground Truth作为监督信号的，只有 Questions/Weak Signal/Environment Access
- 目前来说并不是一个完全和Self-Evolving概念契合的部分



https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/

- 这三个模块都是自我进化，区别是优化目标不同，共同点是模型都在Loop中自我改进，不断迭代去取得更新的结果
- 有些时候这三个模块之间也是互相作用的，例如优化Kernel：
  - 本身任务是 Artifact 的 Optimization
  - 但是可能需要设备信息的Context
  - 同时 Pre-training需要给它写算子的能力
- 比较有意思的工作：SIA
  - 用一个FeedbackAgent来决定是 Update Harness 还是 Update Weights（通过RL）
  - Benchmark使用的是法律文书分类/CUDA算子优化/单细胞RNA去噪





https://lsl.zone/blog/2026/a-taxonomy-of-self-evolving-agents/

和Infra RSI相关工作

- AlphaEvolve: Gemini 代码进化流程 https://arxiv.org/pdf/2506.13131
- First Steps Toward Automated AI Research: https://www.recursive.com/articles/first-steps-toward-automated-ai-research
- KernelEvolve: https://arxiv.org/pdf/2512.23236
- AccelOpt: https://arxiv.org/pdf/2511.15915
- Asari AI: https://asari.ai/blog/inference-optimization
- 智谱 RSI Infra: Toward Recursive Self-Improvement: How GLM Built Its Own Inference Infrastructure

AlphaEvolve

- LLM 提出代码变体 → 自动评测 → 种群选择 → 保存结果到Database → 再修改
- 相关任务：

项目	Gemini 矩阵乘 kernel	FlashAttention 相关代码
修改对象	Tiling Heuristic （降低搜索成本）	XLA 生成的 IR，包含 kernel 与前后处理
优化目标	真实 TPU 上，不同输入形状的实际运行时间	特定 GPU 推理配置下，整个模块的执行时间
正确性依据	保持数学运算不变，通过限制修改对象维持正确性	每次修改都与原代码在随机输入上做数值对照
泛化或最终验证	真实用户输入形状一半用于搜索，另一半用于评估	作者报告最终代码由专家确认对所有输入正确

矩阵乘优化带来平均 23% kernel 加速、约 1% Gemini 训练时间缩减；FlashAttention 案例最高约 32.5% 加速

- 困难点：
  - 难以评测什么时候应该【**停止探索**】？（无限扩张）→ 用计算预算来约束，提前剪枝
  - 设计平衡Exploration和Exploitation时间的**数据库**
  - 在确定了最好的Program之后继续Explore的能力 → 根据程序效果和多样性筛选

https://arxiv.org/pdf/2506.13131

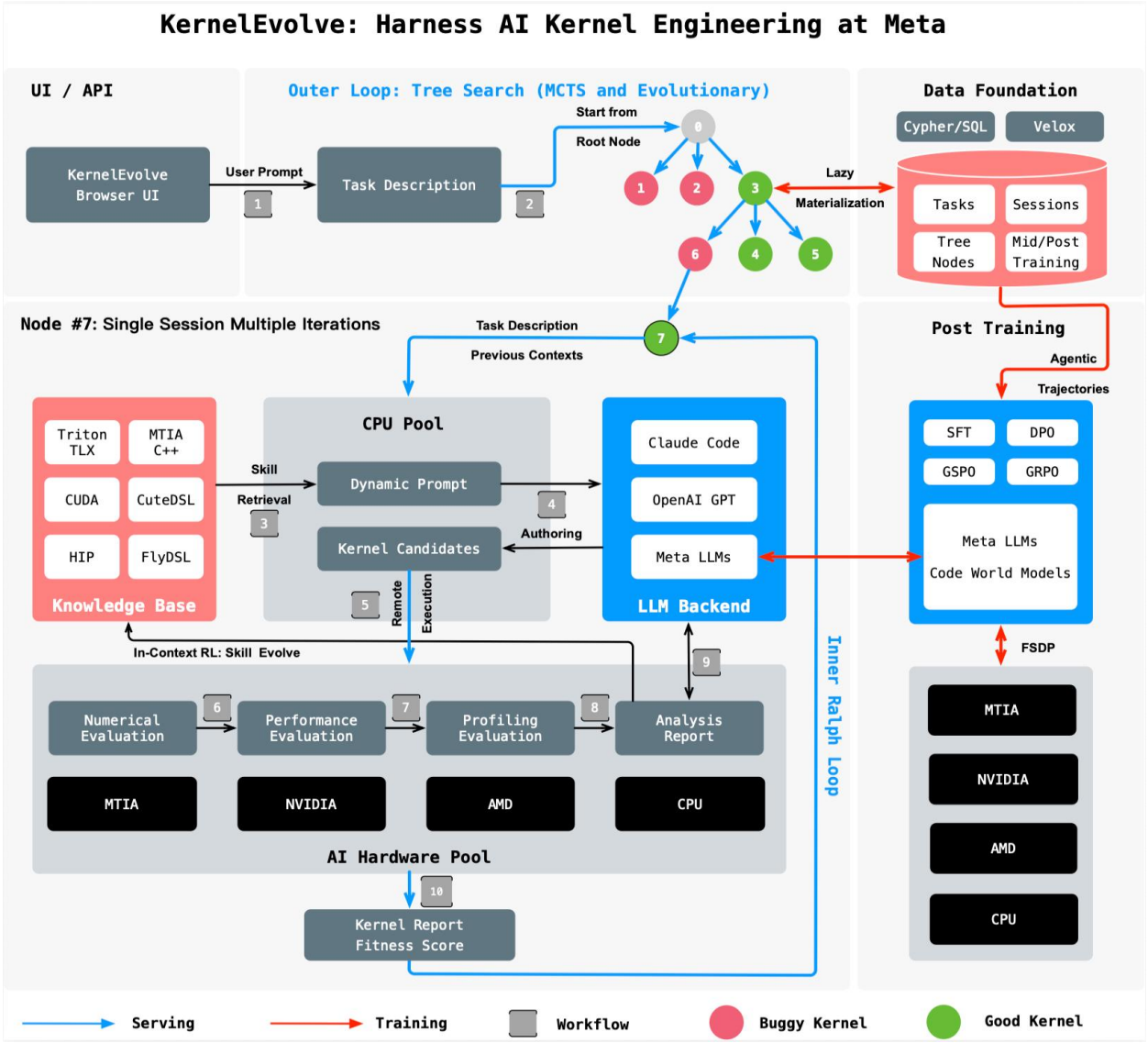
First Steps Toward Automated AI Research

- 测评任务：
  - NanoChat（单张 B200、5 分钟内获得更好的模型）
  - NanoGPT（8 张 H100 上，尽快达到规定验证 loss）
  - SOL（优化 235 个 GPU Kernel）
- 迭代内容：架构、短上下文记忆、辅助损失、注意力机制、Optimizer、Weight-Decay、编译器设置、Fused Kernel
- 任务效果：NanoGPT 训练耗时 79.7 s → 77.5 s，SOL 分数 0.699 → 0.754
- 困难：Reward Hacking （确保系统解决的是预期任务，而不是利用那些表面上符合任务要求且得分很高，但却违背任务初衷的漏洞）→ 使用人工/AI来优化**【作弊检查器】**

https://www.recursive.com/articles/first-steps-toward-automated-ai-research

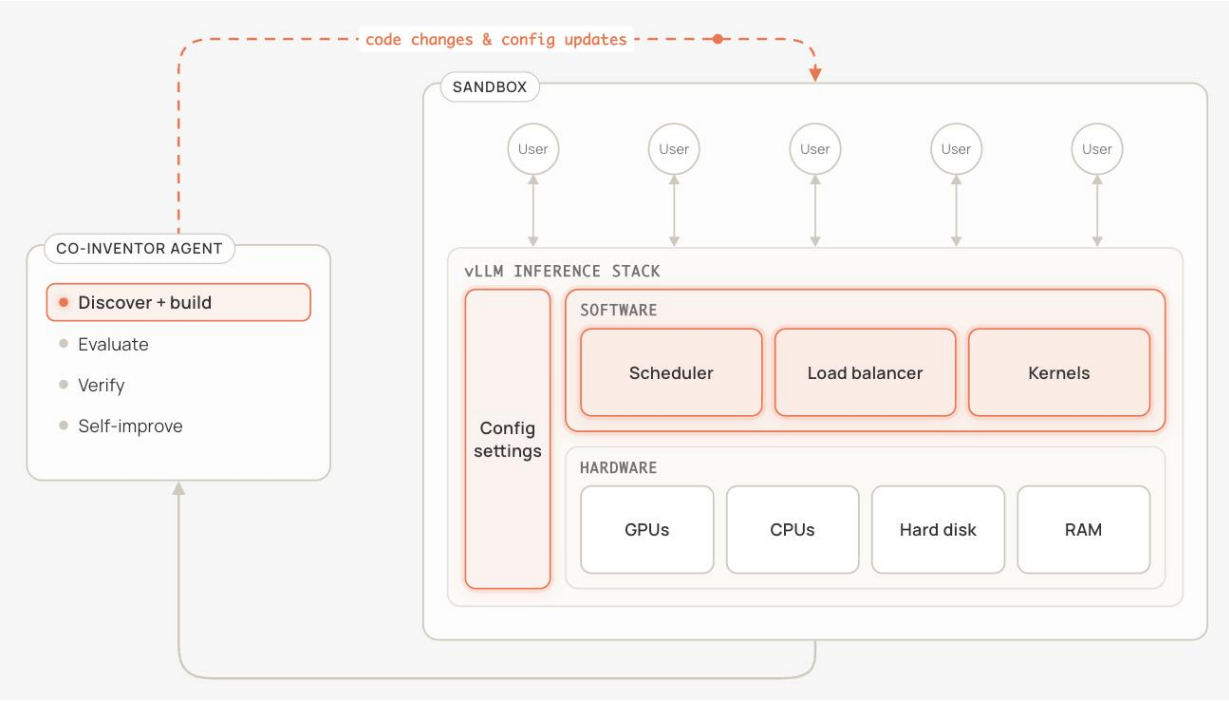
KernelEvolve

- 针对不同环境 MTIA / AMD / NVIDIA / CPU，补齐新芯片上的高效长尾算子
- 主要方案：
  - **【树搜索】**候选
  - 通过Agent检索**【硬件知识库】**来迭代
  - 用精度、耗时和 Profiling 反馈筛选
  - 成功优化写回知识库
- 困难：新芯片知识缺失，硬件差异大
  编译和评测成本高，经验需要跨任务复用
- 广告模型训练吞吐提升 >25%（MTIA）
  推理吞吐提升 >60%（NVIDIA）



Asari AI

- 已有 vLLM v0.23 服务，DeepSeek v4 Pro / GLM 5.2，8 张 B200
- 目标：优化整个推理栈，而非优化单个算子，兼顾吞吐和交互速度，保持模型输出分布
- **【Discover + Build】** → Evaluate → Verify → Self-Improve

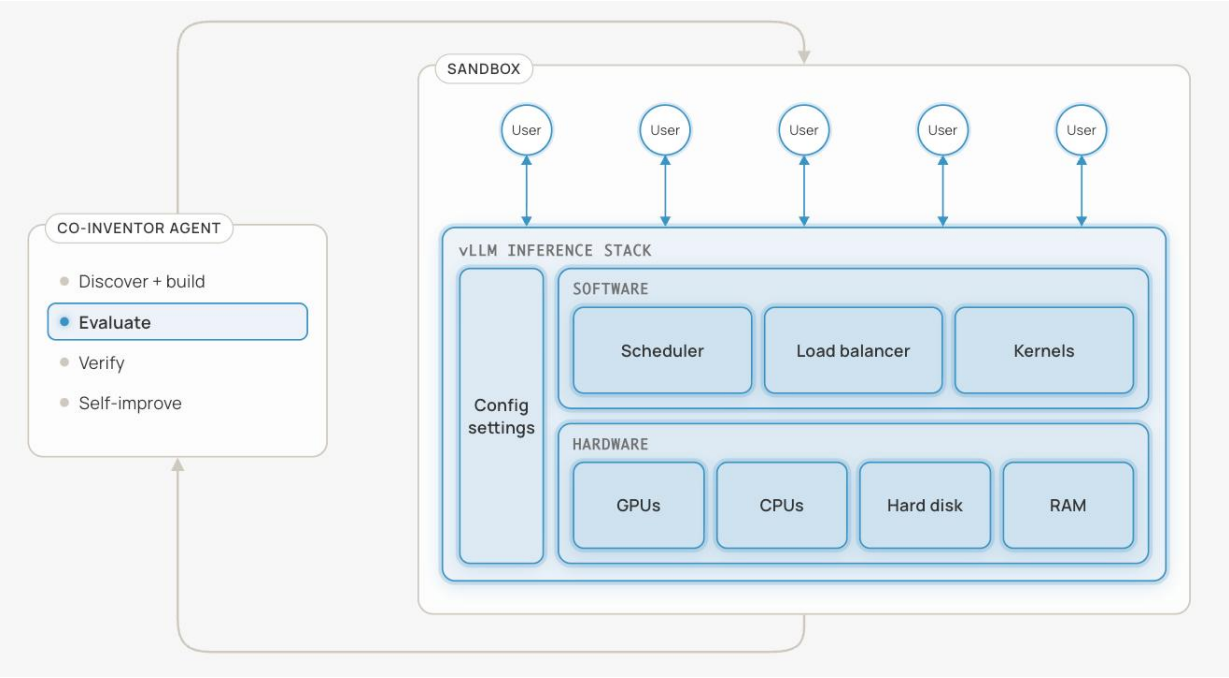


利用动态分析、静态分析和通过自我改进机制学习到的经验，提出优化策略

https://asari.ai/blog/inference-optimization

Asari AI

- 已有 vLLM v0.23 服务，DeepSeek v4 Pro / GLM 5.2，8 张 B200
- 目标：优化整个推理栈，而非优化单个算子，兼顾吞吐和交互速度，保持模型输出分布
- Discover + Build → **【Evaluate】** → Verify → Self-Improve

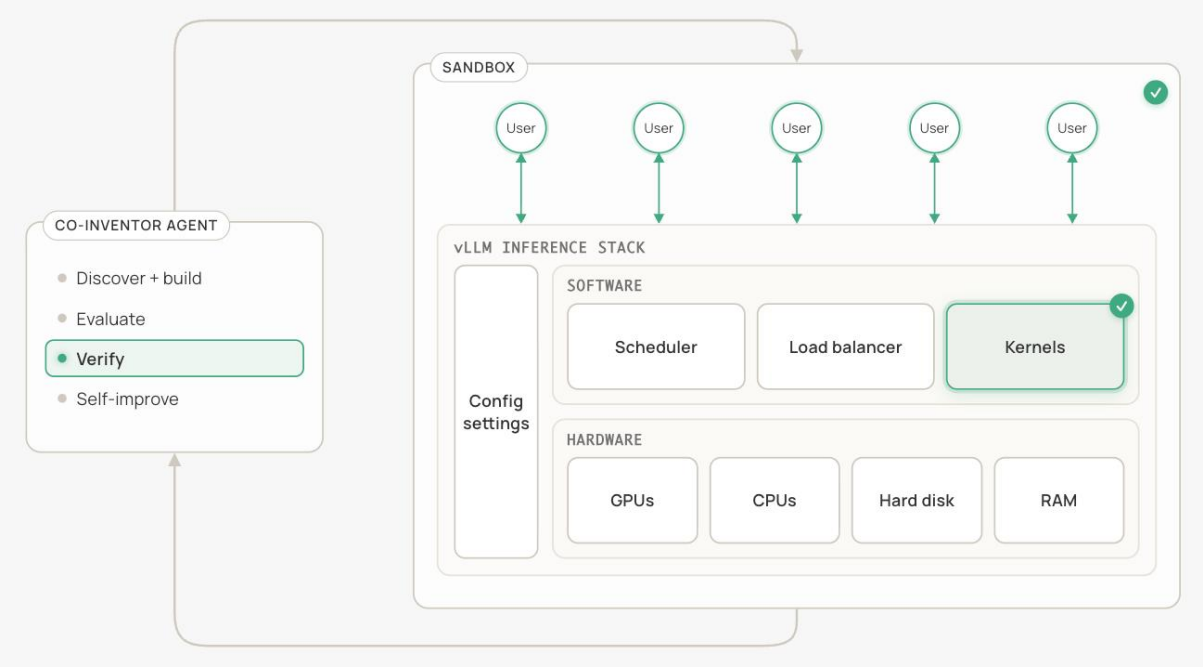


沙箱里面部署并测试，测试的结果通常来说会有很多浮动，需要在这个过程当中定位噪声来源，并且尽可能减少测量误差

https://asari.ai/blog/inference-optimization

Asari AI

- 已有 vLLM v0.23 服务，DeepSeek v4 Pro / GLM 5.2，8 张 B200
- 目标：优化整个推理栈，而非优化单个算子，兼顾吞吐和交互速度，保持模型输出分布
- Discover + Build → Evaluate → **【Verify】** → Self-Improve

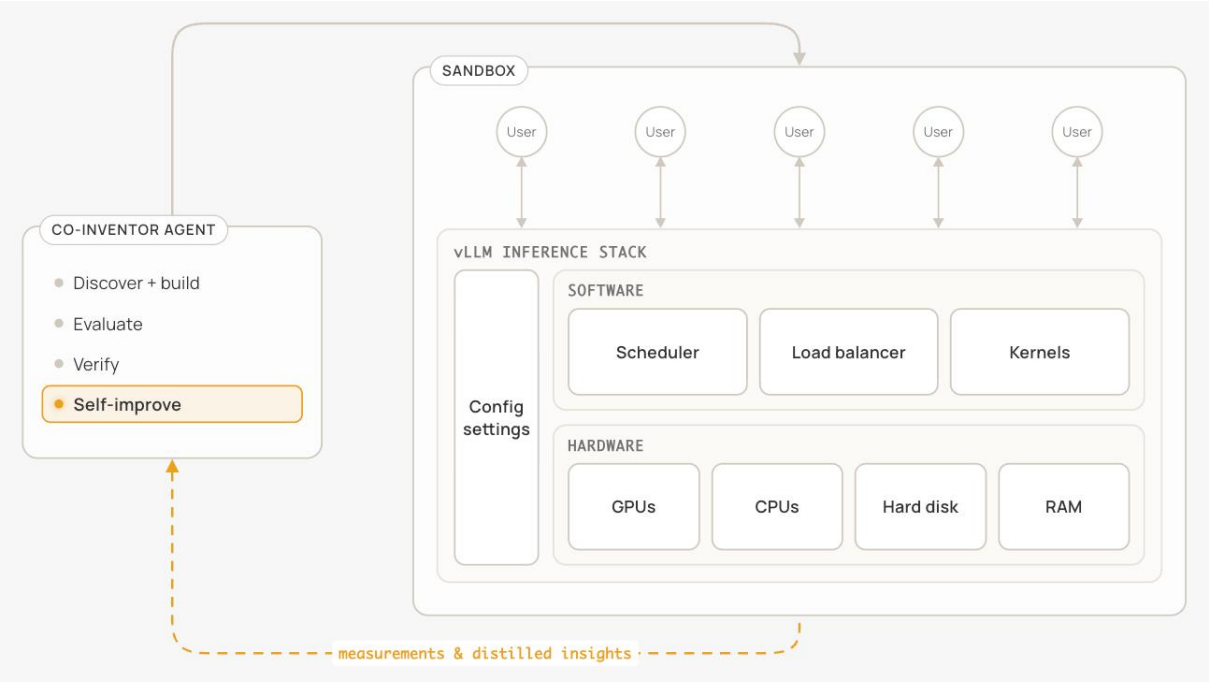


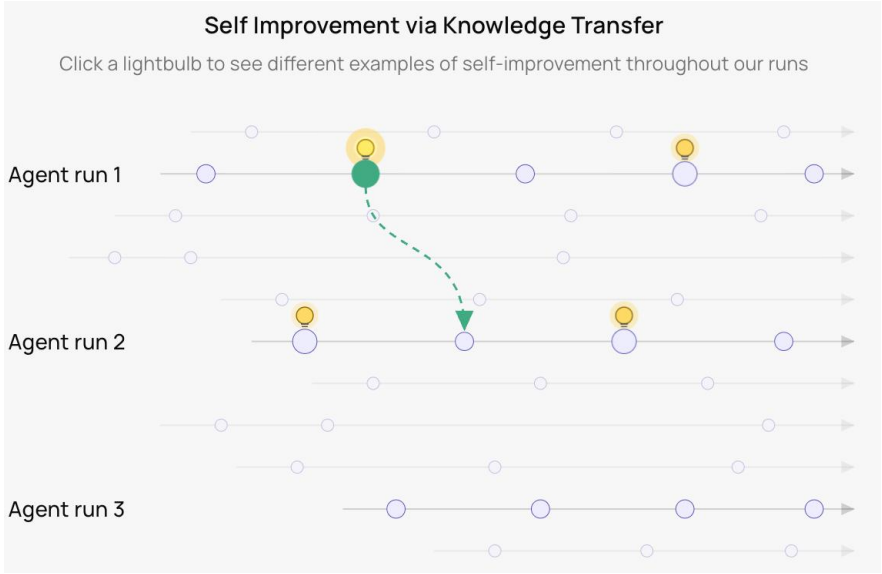
要进行正确性检验，测评Benchmark不能过大（会导致资源浪费）也不能太小（会导致测评结果并不Trustworthy）

https://asari.ai/blog/inference-optimization

Asari AI

- 已有 vLLM v0.23 服务，DeepSeek v4 Pro / GLM 5.2，8 张 B200
- 目标：优化整个推理栈，而非优化单个算子，兼顾吞吐和交互速度，保持模型输出分布
- Discover + Build → Evaluate → Verify → **【Self-Improve】**



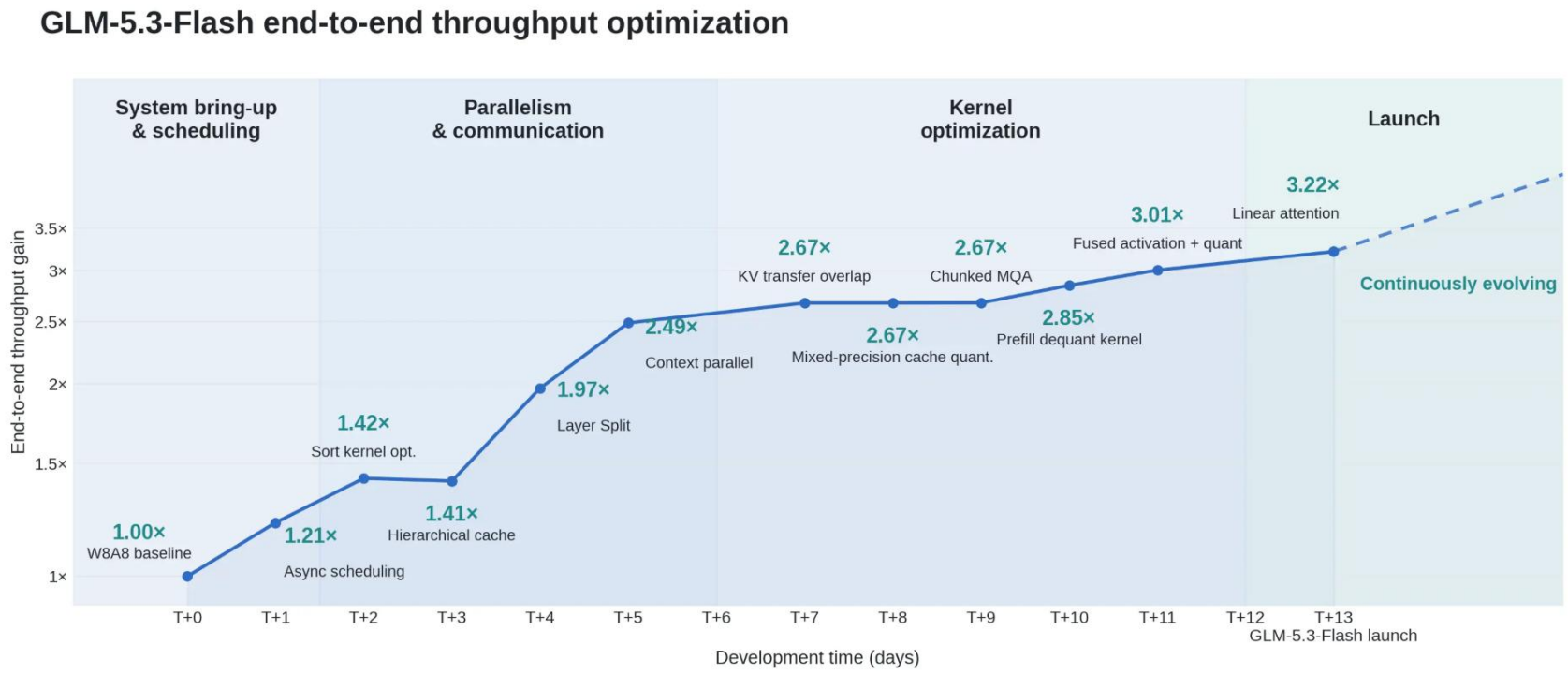


已经出现的问题要沉淀在知识库里面

https://asari.ai/blog/inference-optimization

智谱 RSI Infra

- 国产加速器上的初始模型适配，工程师定义目标、系统边界和反馈环境
- 目标：把 GLM-5.3-Flash 从首次跑通，推进到可承载生产流量的推理服务



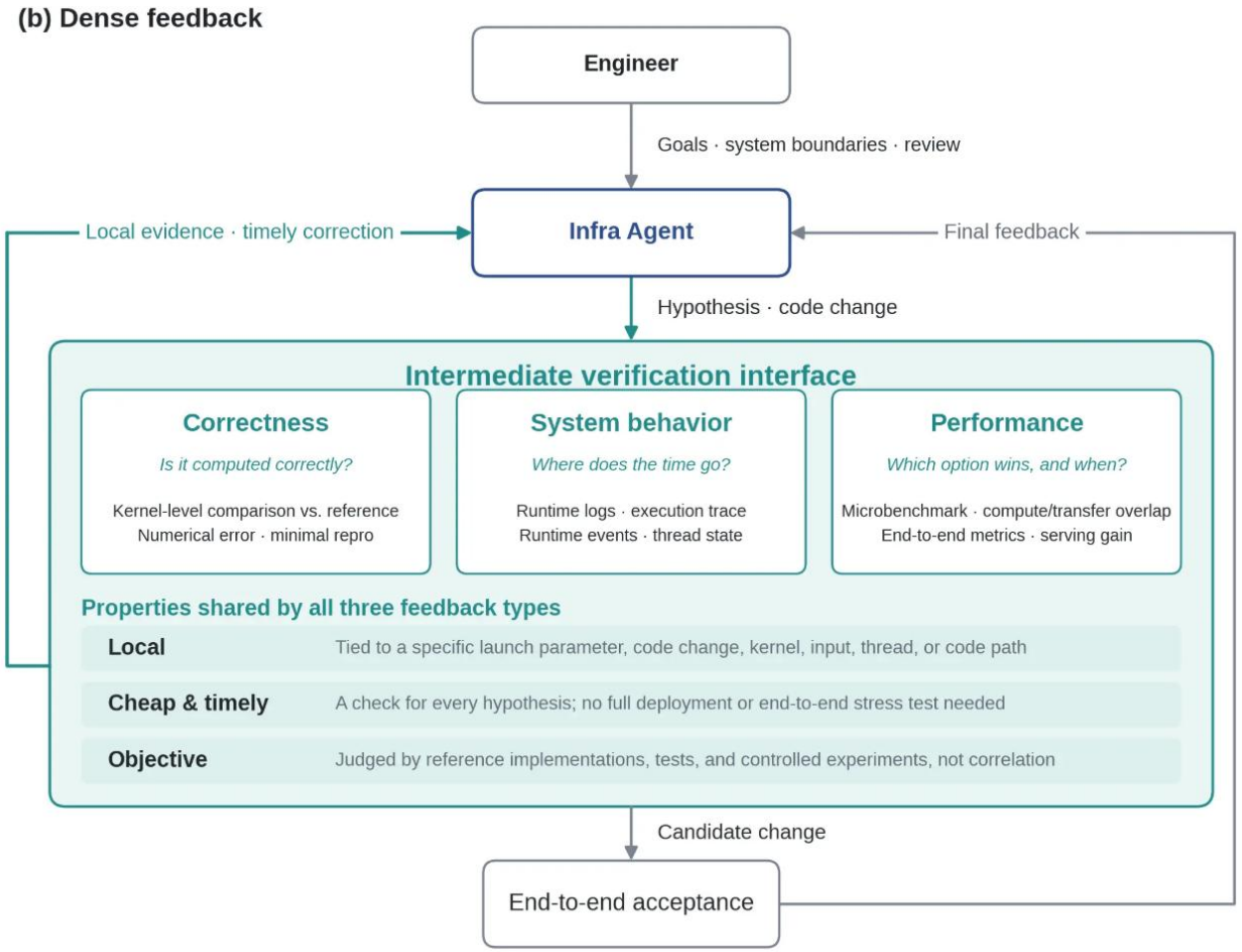
最终版本迭代：

1. 用于线性注意力机制和LM Head的节点内张量并行
2. ReplaySSM
3. W8A8量化
4. 使用INT8/FP8/BF16的混合精度缓存量化以及层分割
5. 引入了编码-预填充-解码（EPD）解耦架构。

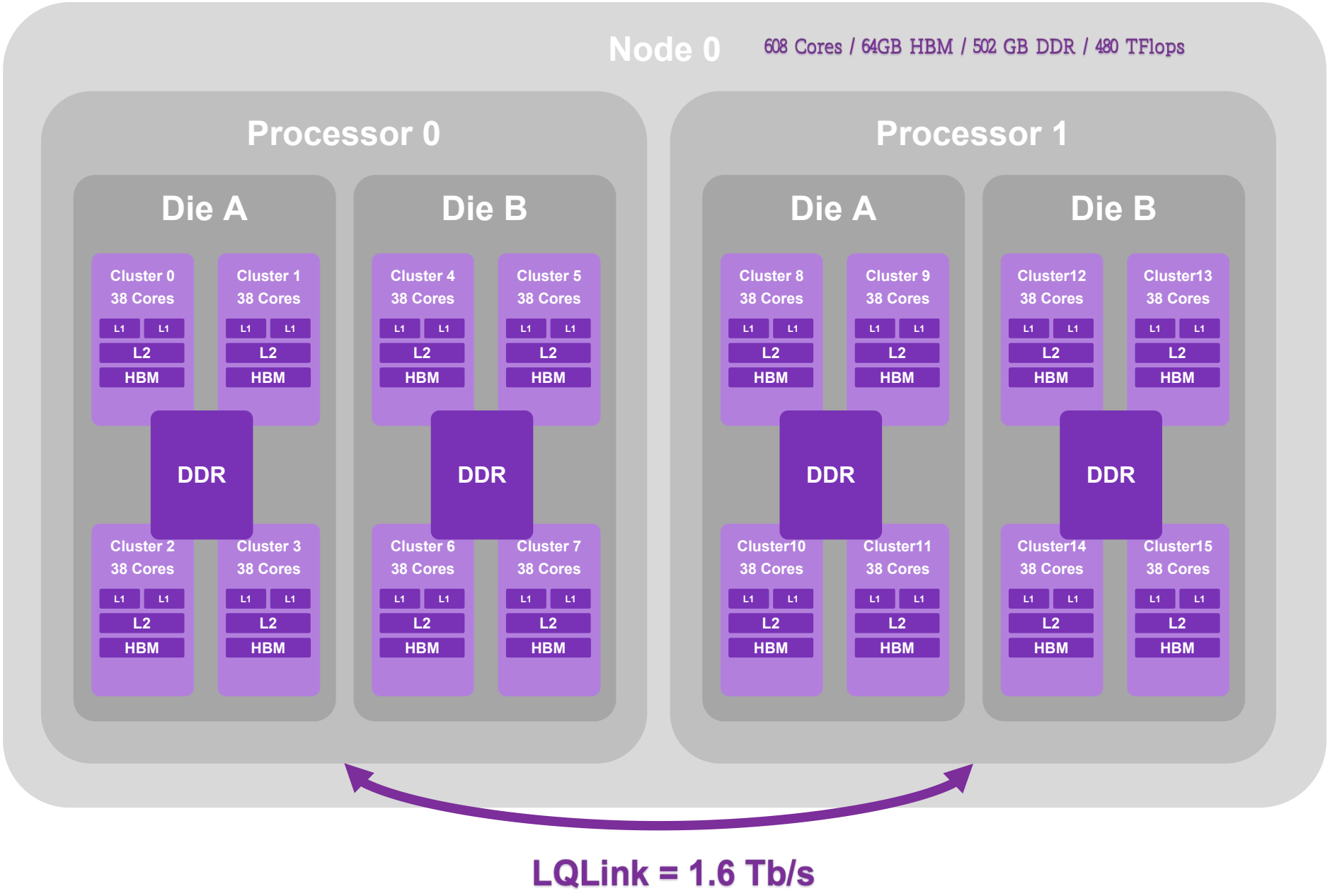
https://z.ai/blog/glm-built-its-inference-infrastructure

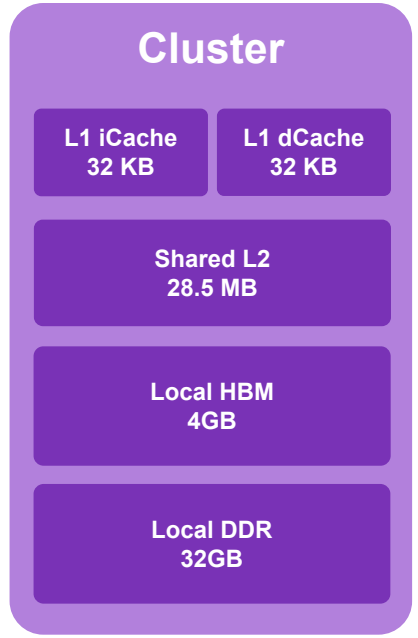
智谱 RSI Infra

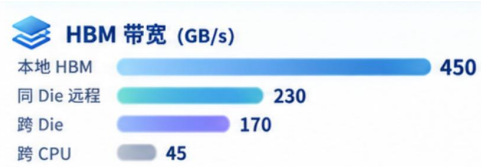
- 回馈：Local / Cheap & Timely / Objective
- 正确性校验
  - 需要建立推理引擎并行策略到内核实现的映射
  - 识别不同并行配置涉及哪些内核以及如何划分输入
- 系统行为反馈
  - 明确验收指标
  - 设置【对照实验】定位异常，Agent根据逐层反馈定位系统问题
- 性能反馈
  - 已有代码提供优化方向，运行反馈决定是否有效，经验回写降低下一轮优化成本



https://z.ai/blog/glm-built-its-inference-infrastructure









DDR 基本没有同 Die 远程访问差异

目标和困难

- 超智融合平台搭载国产Arm 处理器 LX2，搭载国产操作系统 Kylin Linux
- 目标：
  - Artifacts Iterative Optimization
  - 对集群上面部署的Torch系统进行优化，提升训练和推理性能（后续可能会考虑RL等后训练框架）
- 困难：
  - **【生态不完全】**没有一套类似于CUDA的Kernel算法可以直接调用，需要定位是在 调用层、编译层、代码生成层的哪个阶段出现的差异（需要去优化哪个部分）
  - **【硬件知识不齐】**需要在前期迭代过程中沉淀硬件相关知识库
  - **【集群结构与传统GPU不同】**需要重新设计分布式算法，合理利用多层级内存和计算资源
  - **【Micro Bench缺失】**需要先确定下来最小的MicroBench能够快速测试出某个Kernel/算法在集群上面的迭代优化效果（确保精度、速度、系统等待时间等等不同指标）
  - **【平台噪声】**在测试过程当中会有很多性能浮动的地方，要如何剔除掉平台测评带来的噪音

现有结果与计划实验

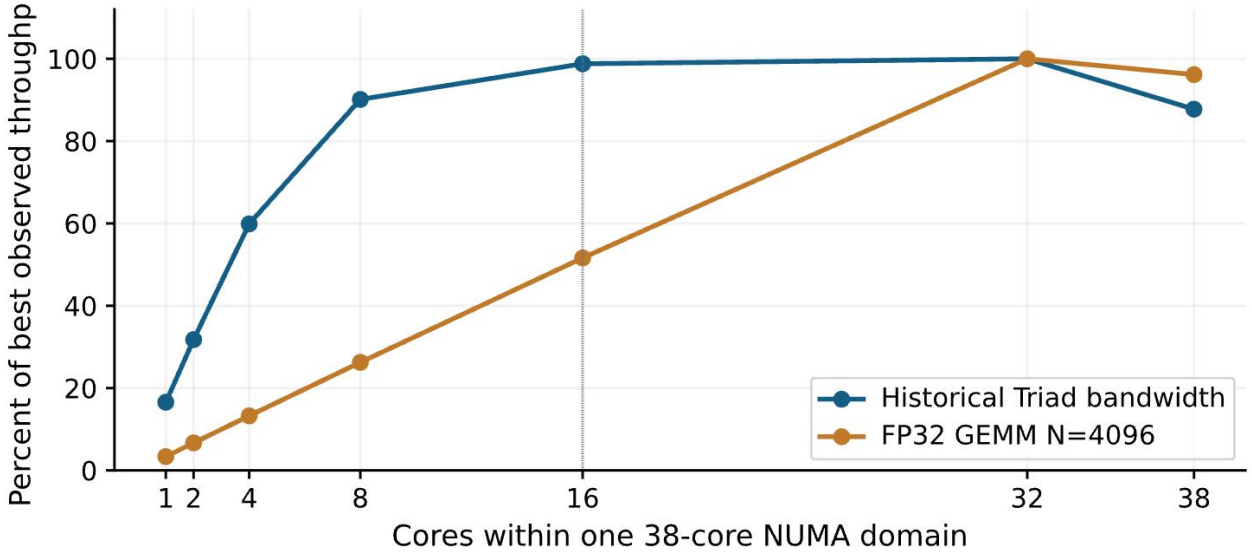
- 目前来说，对于集群的了解还是比较少，所以要通过Profile的方式对集群有一个基础画像，同时需要弄清楚需要对于不同精度的计算支持如何
- **实验模型：** Qwen3-0.6B
- **实验一：参数搜索**
  - 扫描：batch {1, 4, 16, 64, 256} × 线程数 {16, 64, 128, 256} × 精度 {BF16, FP16、FP32}
  - 产出：MFU / 实测算术强度 vs batch 曲线、线程数 × batch 最优组合、逐模块分解数据
  - 锁定 2 个 Batch + 1 个线程数
- **实验二：确定性校验**
  - 参数锁定后：同节点、同环境，重跑 ⩾3 次，5 步 loss 与 grad norm bitwise 全等
- **实验三：AI4AI 最小优化闭环**
  - 候选表带逐模块耗时分解（前向 / 后向分开 × attention / SwiGLU / embedding / RMSNorm / optimizer step），预期热点在 attention 与 SwiGLU，占比 × 可优化性决定优化优先级
  - 验收：至少一个真实优化完整走完闭环，MFU 提升可复现，最高可以到40%，两个 batch 分别验收

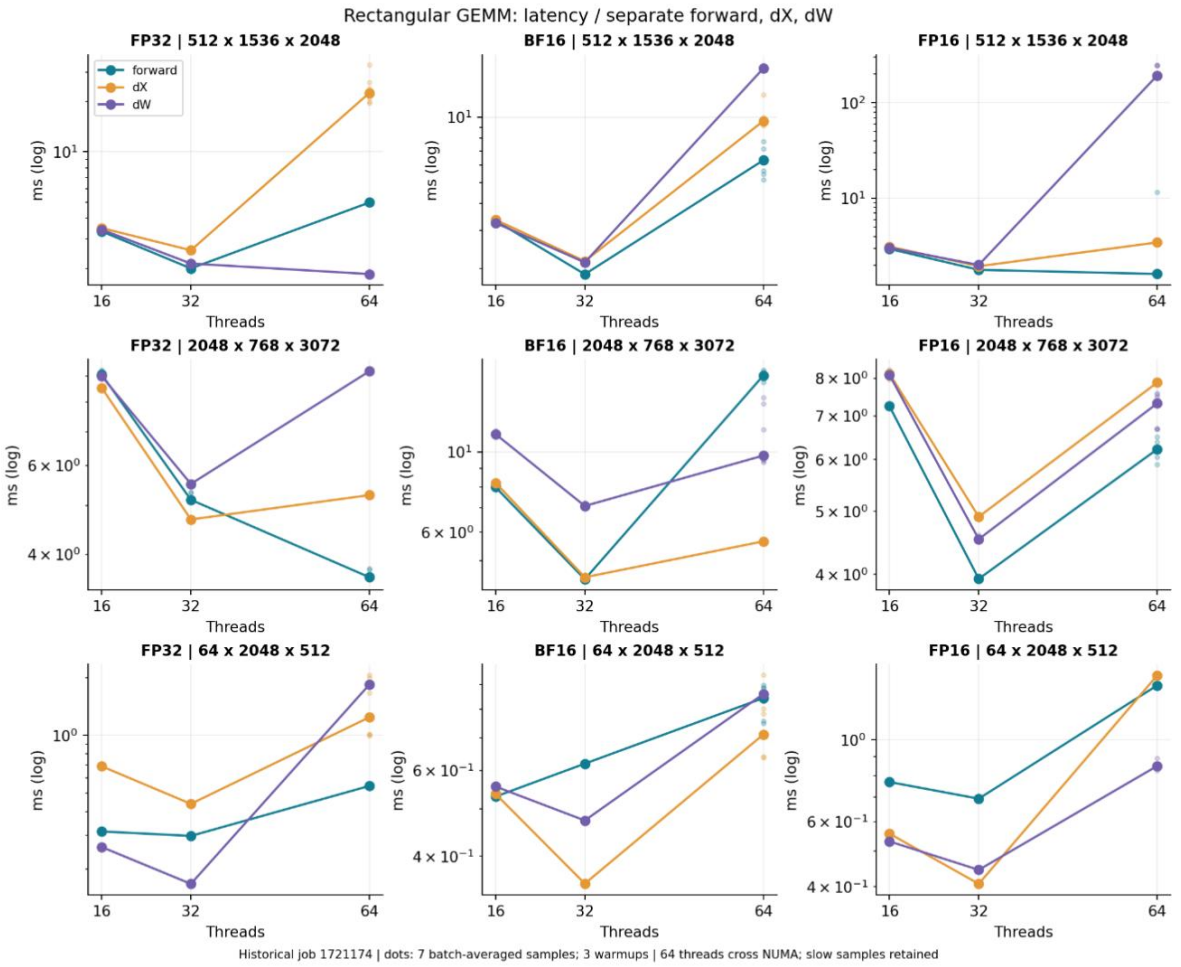

目标与约束
训练 / 推理工作负载
正确性、性能、内存与资源成本
资源权限、预算与停止条件
性能分析
模型 → 组件 → 算子
前向 / 后向、GEMM、内存与线程等待
定位瓶颈、库实现、编译、拓扑与通信瓶颈
瓶颈证据
候选与产物生成
实测热点驱动，提出单因素假设
运行配置、代码补丁、编译与构建产物
冻结版本、关联父 Baseline、支持回退
候选 Artifact
分层评测
MicroBench → 模块 → 端到端
正确性 Gate、同节点 A/B/A 对照
噪声判定、独立复验、资源代价检查
微基准加速
不等于
端到端收益
更新基线，进入下一轮
接纳与交付
可信端到端收益
→ 接纳为新 Baseline
无收益 / 失败 / 证据不足
→ 分类归档
交付：GEMM耗时与吞吐图、
组件前后向图、逐版本收益记录
证据不足
不等于
优化失败
读取知识 ↔ 回写证据
读取知识 ↔ 回写证据
共享支撑层，硬件与优化知识库
硬件拓扑与可用能力
实际后端与适用条件
原始测量与版本证据
成功与失败经验

预计成本申请

- 超智灵晟平台？元（未知成本）
- Codex / GLM Coding Agent 的Token使用大概 5k元

现有结果与计划实验





- 线程扩展效果有限，16线程达到内存访问峰值，32线程达到计算效率峰值，无法利用到38线程
- 不同大小、不同精度GEMM也基本在32线程达到最快计算速度（内核态开销与线程同步等待随线程数增多而放大）

现有结果与计划实验

- 跨Die、跨CPU通信效率低，优先应该保证同Die内访问

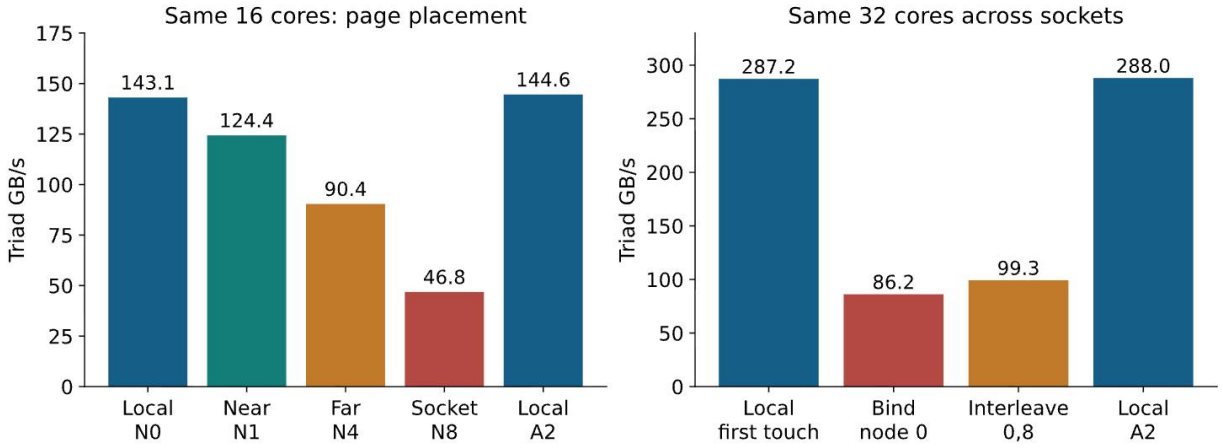


图 3　实测页放置对照。左：固定同一组 16 核；右：固定跨 CPU 的 32 核。逻辑 Triad 读写带宽，原始任务 1709543。

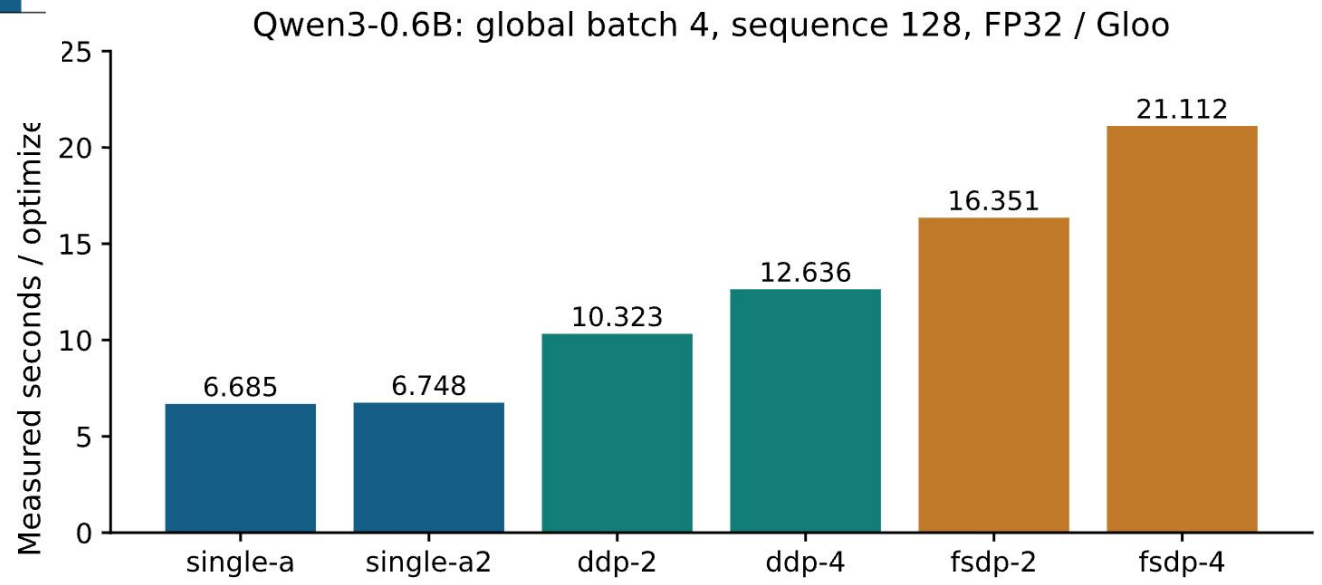


图 8　已有同 allocation 的 Qwen3–0.6B 强扩展。每节点 16 核；20 步，2 步预热，余下 18 步的最大 rank 耗时取中位数。A/A2 包围 DDP 段；FSDP 在 A2 之后。

现有结果与计划实验

$$\text{MFU} = \frac{\text{Model FLOPs per iteration}}{\text{GPU Peak FLOPS} \times \text{World Size} \times \text{Iteration Time}}$$

- MFU 的结果非常低，优化层可能要从算子层优化起

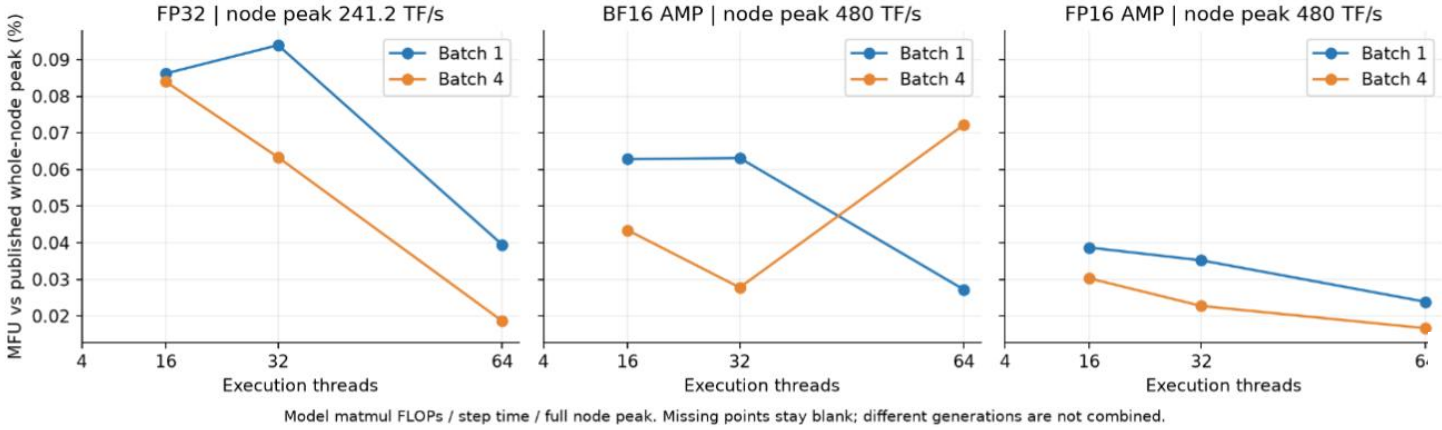
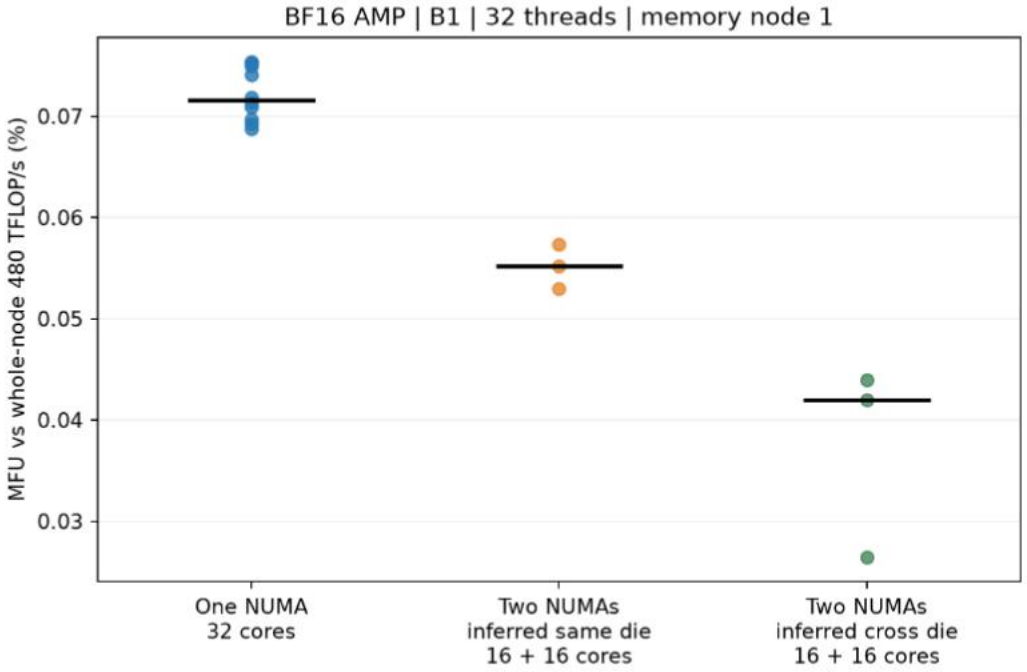


图 4　历史 18 格 MFU，4 线程暂缺

- 跨NUMA/跨Die后MFU会进一步降低



现有结果与计划实验

- Attention & SwiGLU会成为热点，可能接下来会从这个部分进行优化

